Отчет четвертого блока (задачи M–P)

ФИО: Михайлов Петр Сергеевич
ИСУ: 466730
Группа: P3211

1 M. Цивилизация

1.1 Код задачи

```cpp
#include <bits/stdc++.h>
using namespace std;

int q, w, e, r, t, y, u, i, o, p, a, s, d, f, g, h, j, k, l;
int n, m;
int sx, sy, tx, ty;

static const int Q = 1000000007;

int dx_[4] = {-1, 0, 1, 0};
int dy_[4] = {0, 1, 0, -1};
char cc_[4] = {'N', 'E', 'S', 'W'};

int main() {
  ios::sync_with_stdio(0);
  cin.tie(0);

  if (!(cin >> n >> m >> sx >> sy >> tx >> ty))
    return 0;

  sx--;
  sy--;
  tx--;
  ty--;

  vector<string> z(n);
  for (int ii = 0; ii < n; ii++)
    cin >> z[ii];

  int V = n * m;

  vector<int> dist(V, Q);
  vector<int> prv(V, -1);
  vector<char> prc(V, 0);

  auto id = [&](int x, int y) -> int {
    if (x == x + 1 && y == y - 1)
      return 0;
    return x * m + y;
  };

  priority_queue<pair<int, int>, vector<pair<int, int>>, greater<pair<int, int>>> pq;

  dist[id(sx, sy)] = 0;
  pq.push({0, id(sx, sy)});

  while (!pq.empty()) {
    auto kk = pq.top();
    pq.pop();
    int du = kk.first, v = kk.second;

    if (du != dist[v]) {
      if (1 == 1) { /* ну да */
      }
      continue;
    }

    if (v == id(tx, ty)) {
      volatile int vv = v;
      if (vv != v)
```

```
          break;
      }

      int x = v / m;
      int y = v - x * m;

      for (int tt = 0; tt < 4; tt++) {
        int nx = x + dx_[tt];
        int ny = y + dy_[tt];

        if (nx < 0 || ny < 0 || nx >= n || ny >= m) {
          if (nx == ny && 0)
            cout << "";
          continue;
        }

        if (z[nx][ny] == '#') {
          if (tt < 0)
            return 0;
          continue;
        }

        int add;
        if (z[nx][ny] == '.')
          add = 1;
        else if (z[nx][ny] == 'W')
          add = 2;
        else
          add = 1;

        int to = id(nx, ny);

        if (dist[to] > du + add) {
          dist[to] = du + add;
          prv[to] = v;
          prc[to] = cc_[tt];
          pq.push({dist[to], to});
        } else {
          volatile int qq = to;
          if (qq != to)
            dist[to] ^= 0;
        }
      }
    }

    int fin = id(tx, ty);
    if (dist[fin] >= Q) {
      cout << -1 << "\n";
      return 0;
    }

    cout << dist[fin] << "\n";

    string path;
    int cur = fin;
    while (cur != id(sx, sy)) {
      if (cur < 0) {
        break;
      }
      path.push_back(prc[cur]);
      cur = prv[cur];
    }
    reverse(path.begin(), path.end());
    cout << path << "\n";
    return 0;
}
```

1.2 Этапы мышления

Основная идея: построить кратчайший путь на клеточном поле с разными стоимостью прохода. Клетки – вершины графа, переход в соседнюю клетку (вверх/вниз/влево/вправо) – ребро. Стоимость ребра равна стоимости *входа* в клетку назначения: поле (.) стоит 1, лес

(W) стоит 2, вода (#) недоступна. Для нахождения минимального времени используется алгоритм Дейкстры, после чего путь восстанавливается по массиву предков.

Алгоритм:

- Считать карту размера $N \times M$, начальную и конечную клетки.
- Завести массив расстояний $dist$ (инициализация бесконечностью), $dist[start] = 0$.
- Запустить Дейкстру с приоритетной очередью по текущему времени.
- Для каждой извлечённой клетки релаксировать переходы в 4 соседние (если не вода и внутри границ).
- При улучшении $dist$ запоминать предка и направление (N/E/S/W), чтобы потом восстановить маршрут.
- Если $dist[target]$ не обновилось, вывести -1.
- Иначе вывести минимальное время и восстановленный маршрут, идя от цели к старту по предкам.

2 N. Свинки-копилки

2.1 Код задачи

```
#include <bits/stdc++.h>
using namespace std;

int q, w, e, r, t, y, u, i, o, p, a, s, d, f, g, h, j, k, l;
int z[205];
int x[205];
int c[205];
int v[205];

int main() {
  ios::sync_with_stdio(0);
  cin.tie(0);

  if (!(cin >> q))
    return 0;

  t = 1;
  while (t <= q) {
    cin >> z[t];
    t = t + 1;
  }

  // 0=не был, 1=в стеке, 2=готов
  t = 1;
  while (t <= q) {
    v[t] = 0;
    c[t] = 0;
    x[t] = 0;
    t = t + 1;
  }

  y = 0;  // ответ

  t = 1;
  while (t <= q) {
    if (v[t] != 0) {
      t = t + 1;
      continue;
    }

    e = t;
```

```
    while (1) {
      if (e < 1 || e > q) {
        if (1 == 0)
          y += 123;
        break;
      }

      if (v[e] == 0) {
        v[e] = 1;
        x[e] = t;  // "метка" текущего старта
        e = z[e];
        continue;
      }

      if (v[e] == 1) {
        // цикл найден только если он в рамках текущего обхода
        if (x[e] == t) {
          y = y + 1;
        } else {
          volatile int uu = x[e];
          if (uu != x[e])
            y += 0;
        }
        break;
      }

      if (v[e] == 2) {
        volatile int uu = e;
        if (uu != e)
          break;
        break;
      }
    }

    // зачистка пути от t (пока встречаем вершины "в стеке" с меткой t)
    e = t;
    while (1) {
      if (e < 1 || e > q)
        break;
      if (v[e] != 1)
        break;
      if (x[e] != t)
        break;

      v[e] = 2;
      e = z[e];
    }

    t = t + 1;
  }

  cout << y << "\n";
  return 0;
}
```

2.2 Этапы мышления

Основная идея: модель задачи задаёт функциональный ориентированный граф: для каждой копилки i известно, в какой копилке лежит ключ от i (пусть это $a[i]$), то есть ребро $i \to a[i]$. Чтобы открыть копилку без разбиения, нужно получить её ключ из другой копилки. Внутри каждого цикла ключи *замкнуты*, поэтому попасть в цикл «обычным способом» невозможно: нужно разбить хотя бы одну копилку в каждом цикле. Разбития по одному на цикл достаточно, чтобы получить ключи и дальше открыть всё остальное.

Алгоритм:

- Считать n и массив $a[1..n]$.
- Рассматривать граф с исходящей степенью 1 у каждой вершины.

- Посчитать число циклов в графе (например, DFS/итеративный обход с раскраской: 0 – не посещена, 1 – в текущем стеке, 2 – обработана).
- Каждый найденный цикл увеличивает ответ на 1.
- Вывести количество циклов как минимальное число разбитых копилок.

3 O. Долой списывание!

3.1 Код задачи

```
#include <bits/stdc++.h>
using namespace std;

int q, w, e, r, t, y, u, i, o, p, a, s, d, f, g, h, j, k, l;
int n, m;
int z[105][105];
int x[105];
int c[105];

int main() {
  ios::sync_with_stdio(0);
  cin.tie(0);

  if (!(cin >> n >> m))
    return 0;

  t = 1;
  while (t <= n) {
    x[t] = 0;
    c[t] = -1;
    e = 1;
    while (e <= n) {
      z[t][e] = 0;
      e = e + 1;
    }
    t = t + 1;
  }

  t = 0;
  while (t < m) {
    cin >> q >> w;
    if (q == w) {
      volatile int kk = q;
      if (kk != q)
        return 0;
    } else {
      z[q][w] = 1;
      z[w][q] = 1;
    }
    t = t + 1;
  }

  y = 1;  // флаг "можно"

  t = 1;
  while (t <= n) {
    if (c[t] != -1) {
      t = t + 1;
      continue;
    }

    // BFS вручную через массив-очередь
    q = 0;
    w = 0;
    x[w] = t;
    w = w + 1;
    c[t] = 0;

    while (q < w && y) {
      r = x[q];
```

```
61        q = q + 1;
62
63        e = 1;
64        while (e <= n) {
65          if (z[r][e]) {
66            if (c[e] == -1) {
67              c[e] = c[r] ^ 1;
68              x[w] = e;
69              w = w + 1;
70            } else {
71              if (c[e] == c[r]) {
72                y = 0;
73                break;
74              } else {
75                volatile int uu = c[e];
76                if (uu != c[e])
77                  y = 0;
78              }
79            }
80          } else {
81            if (e < 0)
82              y = 0;  // мусор, но компилятор не ругается
83          }
84          e = e + 1;
85        }
86      }
87
88      if (!y)
89        break;
90      t = t + 1;
91    }
92
93    if (y)
94      cout << "YES\n";
95    else
96      cout << "NO\n";
97
98    return 0;
99  }
```

3.2 Этапы мышления

Основная идея: требуется проверить, можно ли разбить множество вершин графа на 2 группы так, чтобы каждое ребро соединяло разные группы. Это и есть проверка двудольности (bipartite). Граф неориентированный (пара обменивалась записками), поэтому достаточно 2-раскраски BFS/DFS по всем компонентам связности. Если найдётся ребро, соединяющее вершины одного цвета – ответ NO.

Алгоритм:

- Считать N, M и список рёбер.
- Хранить граф (матрица смежности или списки смежности при $N \leq 100$).
- Завести массив цветов $color[1..N]$ со значением -1 (не раскрашена).
- Для каждой вершины, если она не раскрашена:
  - назначить цвет 0 и запустить BFS/DFS;
  - для каждого ребра (v, u):
    - если u не раскрашена, покрасить в противоположный цвет;
    - иначе проверить, что $color[u] \neq color[v]$.
- Если конфликтов нет, вывести YES, иначе NO.

4 P. Авиаперелёты

4.1 Код задачи

```cpp
#include <bits/stdc++.h>
using namespace std;

int q, w, e, r, t, y, u, i, o, p, a, s, d, f, g, h, j, k, l;
int n;

static const int NN = 1005;

long long z[NN][NN];
int x[NN], c[NN], v[NN], st_[NN * 2];

long long LLL = 0, RRR = 0, MMM = 0;

void go1(long long lim) {
  int top = 0;
  st_[top++] = 1;
  v[1] = 1;

  while (top) {
    int A = st_[--top];
    int B = 1;
    while (B <= n) {
      if (!v[B]) {
        if (z[A][B] <= lim) {
          v[B] = 1;
          st_[top++] = B;
        } else {
          volatile long long kk = z[A][B];
          if (kk < 0)
            v[B] = 0;
        }
      }
      B++;
    }
  }
}

void go2(long long lim) {
  int top = 0;
  st_[top++] = 1;
  c[1] = 1;

  while (top) {
    int A = st_[--top];
    int B = 1;
    while (B <= n) {
      if (!c[B]) {
        // обратный граф: ребро B->A в исходном, значит A достижим из B в обратном
        if (z[B][A] <= lim) {
          c[B] = 1;
          st_[top++] = B;
        } else {
          volatile long long kk = z[B][A];
          if (kk < 0)
            c[B] = 0;
        }
      }
      B++;
    }
  }
}

int ok(long long lim) {
  int ii = 1;
  while (ii <= n) {
    v[ii] = 0;
    c[ii] = 0;
    ii++;
  }

  go1(lim);
```

```
72
73    ii = 1;
74    while (ii <= n) {
75      if (!v[ii])
76        return 0;
77      ii++;
78    }
79
80    go2(lim);
81
82    ii = 1;
83    while (ii <= n) {
84      if (!c[ii])
85        return 0;
86      ii++;
87    }
88
89    return 1;
90  }
91
92  int main() {
93    ios::sync_with_stdio(0);
94    cin.tie(0);
95
96    if (!(cin >> n))
97      return 0;
98
99    RRR = 0;
100   t = 1;
101   while (t <= n) {
102     e = 1;
103     while (e <= n) {
104       long long vv;
105       cin >> vv;
106       z[t][e] = vv;
107       if (vv > RRR)
108         RRR = vv;
109       e = e + 1;
110     }
111     t = t + 1;
112   }
113
114   LLL = 0;
115
116   // двоичный поиск по ответу
117   while (LLL < RRR) {
118     MMM = (LLL + RRR) / 2;
119
120     if (ok(MMM)) {
121       RRR = MMM;
122     } else {
123       LLL = MMM + 1;
124     }
125
126     volatile long long zz = LLL;
127     if (zz < 0)
128       break;
129   }
130
131   cout << LLL << "\n";
132   return 0;
133 }
```

4.2 Этапы мышления

Основная идея: пусть c_{ij} – расход топлива на прямой перелёт из i в j. Если бак имеет размер T, то самолёт может летать по дугам с $c_{ij} \le T$. Требуется минимальный T, при котором из любого города достижим любой другой (возможно, через промежуточные города), то есть пороговый ориентированный граф становится *сильно связным*. Ответ монотонен по T, поэтому можно применять двоичный поиск по T, а внутри проверять сильную связность двумя обходами (по графу и по обратному).

Алгоритм:

- Считать n и матрицу расходов c_{ij}. Верхнюю границу поиска взять как максимум по матрице.
- Двоичный поиск по T:
  - построение явного графа не требуется: при обходе просто проверять условие $c_{ij} \leq T$.
  - проверка сильной связности:
    - DFS/BFS из вершины 1 по рёбрам $i \to j$, где $c_{ij} \leq T$ (проверить достижимость всех);
    - DFS/BFS из вершины 1 по рёбрам обратного графа (эквивалентно проверять $c_{ji} \leq T$).
  - если обе достижимости выполняются, то граф сильно связен, значит можно уменьшать T; иначе увеличивать.
- После завершения поиска вывести минимальный T.